PLUTO ACADEMY
AI & MACHINE LEARNING

Project 1 and Project 2 — Complete Beginner-Friendly Guide

Project 1: Exploratory Data Analysis (EDA) using Netflix titles data

Project 2: Iris Flower Classification using three machine-learning models

Includes setup instructions, runnable code cells, explanations, six charts, model evaluation, report templates, and submission checklist.

Important: Run every cell in Google Colab. Write your findings using your own actual outputs; do not invent results.

Before you start

1. Open https://colab.research.google.com and sign in. 2. Create a new notebook. 3. For Project 1, download the Netflix CSV from https://www.kaggle.com/datasets/shivamb/netflix-shows and upload **netflix_titles.csv** to Colab. 4. Project 2 uses the Iris dataset built into scikit-learn. 5. Save each notebook and set sharing to “Anyone with the link can view.”

What Pluto Academy expects

Project 1: inspect and clean data, answer five questions, create at least six chart types, write five evidence-based insights, and add a 3–5 line note about the most surprising finding. Project 2: preprocess data, train at least three models, compare classification metrics, show a confusion matrix, and write a five-line conclusion. Both need public GitHub repositories and shareable Colab notebooks.

PROJECT 1 — NETFLIX DATA ANALYSIS

Goal: act as a data analyst. Explore the dataset, find patterns, create charts, and explain what the data shows. This is descriptive analysis, not a prediction model.

Cell 1 — Load and inspect the CSV

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from google.colab import files

uploaded = files.upload()  # Select netflix_titles.csv
df = pd.read_csv("netflix_titles.csv")

print("Rows and columns:", df.shape)
display(df.head())
df.info()
print("\nMissing values before cleaning:")
print(df.isnull().sum())
print("Duplicate rows:", df.duplicated().sum())
```

Explanation: This loads the CSV, displays example rows, checks dataset size and data types, and counts missing values and duplicate rows.

Cell 2 — Clean and prepare the data

```
df = df.drop_duplicates().copy()
df["date_added"] = pd.to_datetime(df["date_added"], errors="coerce")

text_cols = ["director", "cast", "country", "rating",
             "duration", "listed_in", "description"]
for col in text_cols:
    df[col] = df[col].fillna("Unknown")

df["year_added"] = df["date_added"].dt.year
df["duration_number"] = pd.to_numeric(
    df["duration"].str.extract(r"(\d+)")[0], errors="coerce"
)

print("Shape after cleaning:", df.shape)
print("Duplicates remaining:", df.duplicated().sum())
print(df.isnull().sum())
```

Why: duplicates could count titles twice. Missing text is labeled Unknown. Invalid dates stay empty instead of being guessed. Movie duration is in minutes while TV duration is in seasons, so do not compare these units directly.

Cell 3 — Answer five questions

```
# Q1: How many Movies and TV Shows?
print(df["type"].value_counts())

# Q2: Which countries have the most titles?
countries = df[df["country"] != "Unknown"]["country"].str.split(", ").explode()
print(countries.value_counts().head(10))

# Q3: Which content ratings are most common?
print(df["rating"].value_counts().head(10))

# Q4: Which genres/categories appear most often?
genres = df["listed_in"].str.split(", ").explode()
print(genres.value_counts().head(10))

# Q5: How many titles were added each year?
print(df["year_added"].value_counts().sort_index())
```

Interpretation: use the actual output counts in your report. A title may list multiple countries and genres, so these counts can overlap.

PROJECT 1 — SIX VISUALIZATIONS

Run this cell after loading and cleaning the dataset. The charts include titles and labels.

```
# 1. Bar chart: Movies vs TV Shows
df["type"].value_counts().plot(kind="bar", title="Movies vs TV Shows")
plt.xlabel("Content type"); plt.ylabel("Number of titles")
plt.tight_layout(); plt.show()

# 2. Line chart: titles added by year
added = df["year_added"].dropna().astype(int).value_counts().sort_index()
added.plot(kind="line", marker="o", title="Titles Added to Netflix by Year")
plt.xlabel("Year added"); plt.ylabel("Number of titles")
plt.tight_layout(); plt.show()

# 3. Histogram: release-year distribution
df["release_year"].dropna().plot(kind="hist", bins=20, title="Distribution of Release Years")
plt.xlabel("Release year"); plt.ylabel("Frequency")
plt.tight_layout(); plt.show()

# 4. Scatter plot: release year vs year added
tmp = df.dropna(subset=["release_year", "year_added"])
plt.figure(figsize=(7, 4))
plt.scatter(tmp["release_year"], tmp["year_added"], alpha=0.35)
plt.title("Release Year vs Year Added")
plt.xlabel("Release year"); plt.ylabel("Year added")
plt.tight_layout(); plt.show()

# 5. Pie chart: Movies vs TV Shows share
df["type"].value_counts().plot(kind="pie", autopct="%1.1f%%",
                               title="Share of Movies and TV Shows")
plt.ylabel(""); plt.tight_layout(); plt.show()

# 6. Heatmap: numeric correlations
cols = ["release_year", "year_added", "duration_number"]
plt.figure(figsize=(6, 4))
sns.heatmap(df[cols].corr(numeric_only=True), annot=True,
            cmap="coolwarm", vmin=-1, vmax=1)
plt.title("Correlation Heatmap")
plt.tight_layout(); plt.show()
```

Chart guide: bar = category comparison; line = change by year; histogram = distribution; scatter = relationship between two numeric variables; pie = proportions; heatmap = correlation. Correlation does not prove cause and effect.

Project 1 — Report template

Complete these using your outputs:
1. The dataset contains ___ Movies and ___ TV Shows.
2. The most frequently listed country is ___.
3. The most common rating is ___.
4. The most frequent category/genre is ___.
5. The year with the most titles added was ___.
Surprising finding (3–5 lines): I was surprised that ___. The chart/output showed ___. This may matter because ___. One limitation is ___.

Five-line report structure

1. This project analyzed the Netflix titles dataset. 2. I removed duplicates and handled missing text values. 3. I compared title types, countries, ratings, categories, and years. 4. The charts showed patterns in the dataset. 5. These findings describe the dataset and do not prove why Netflix made a decision.

PROJECT 2 — IRIS FLOWER CLASSIFICATION

Goal: predict one of three Iris species from four flower measurements. This is supervised classification; the dataset is included with scikit-learn.

Cell 1 — Load data and split 80/20

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
    f1_score, classification_report, ConfusionMatrixDisplay)

iris = load_iris(as_frame=True)
X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)
print("Training rows:", len(X_train), "Testing rows:", len(X_test))
print("Features:", list(X.columns))
print("Species:", list(iris.target_names))
display(X.head())
```

Explanation: X contains the four measurements and y contains the known species. The model learns from 80% of rows and is evaluated on the remaining 20%. random_state makes the split repeatable; stratify keeps class proportions similar.

Cell 2 — Train and compare three models

```
models = {
    "Logistic Regression": make_pipeline(
        StandardScaler(), LogisticRegression(max_iter=1000)),
    "K-Nearest Neighbors": make_pipeline(
        StandardScaler(), KNeighborsClassifier(n_neighbors=5)),
    "Random Forest": RandomForestClassifier(
        n_estimators=100, random_state=42)
}

results = []
trained = {}
for name, model in models.items():
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    trained[name] = model
    results.append({
        "Model": name,
        "Accuracy": accuracy_score(y_test, pred),
        "Precision (macro)": precision_score(y_test, pred, average="macro", zero_division=0),
        "Recall (macro)": recall_score(y_test, pred, average="macro", zero_division=0),
        "F1 (macro)": f1_score(y_test, pred, average="macro", zero_division=0)
    })

results_df = pd.DataFrame(results).sort_values("Accuracy", ascending=False)
display(results_df.round(3))
```

Metric meanings: Accuracy is the share of all predictions that are correct. Precision measures how often a predicted class is correct. Recall measures how many real examples of a class were found. F1 combines precision and recall. Macro averages the scores equally across the three species. Higher is generally better.

PROJECT 2 — EVALUATION AND CONCLUSION

Cell 3 — Best model and confusion matrix

```
best_name = results_df.iloc[0]["Model"]
best_model = trained[best_name]
best_pred = best_model.predict(X_test)

print("Best model by test accuracy:", best_name)
print(classification_report(
    y_test, best_pred, target_names=iris.target_names, zero_division=0
))

ConfusionMatrixDisplay.from_predictions(
    y_test, best_pred, display_labels=iris.target_names
)
plt.title("Confusion Matrix - " + best_name)
plt.tight_layout(); plt.show()
```

Confusion matrix: rows are the true species and columns are the predicted species. The diagonal cells are correct predictions; off-diagonal cells are mistakes. This notebook chooses the best model by test accuracy, but review the other metrics too.

Cell 4 — Try one prediction

```
example = pd.DataFrame([{
    "sepal length (cm)": 5.1,
    "sepal width (cm)": 3.5,
    "petal length (cm)": 1.4,
    "petal width (cm)": 0.2
}])
predicted = best_model.predict(example)[0]
print("Predicted species:", iris.target_names[predicted])
```

Five-line conclusion template — fill in after running the code

1. I used the Iris dataset to predict species from four measurements.
2. I trained Logistic Regression, K-Nearest Neighbors, and Random Forest.
3. The model with the highest test accuracy was ___, with accuracy ___.
4. The confusion matrix showed ___ correct predictions and ___ errors.
5. Based on this test split, ___ performed best by accuracy; results may change with another split or dataset.

README template for each GitHub repository

```
# Project title

## Objective
Explain the question this project answers.

## Dataset
Project 1: Netflix Movies and TV Shows dataset from Kaggle.
Project 2: Iris dataset included with scikit-learn.

## Tools
Python, Pandas, Matplotlib, Seaborn / scikit-learn.

## Steps
Describe loading, cleaning or preprocessing, analysis/model training, and evaluation.

## Results
Add your actual charts, metrics, and findings.

## How to run
Open the .ipynb notebook in Google Colab and run cells from top to bottom.
```

Final submission checklist

- ■ Both notebooks run from first cell to last without unresolved errors.
- ■ Project 1 includes cleaning notes, five answered questions, six chart types, five insights, and a 3–5 line surprising-finding note.
- ■ Project 2 includes preprocessing explanation, three models, a comparison table, a confusion matrix, and a five-line conclusion.
- ■ Create two separate public GitHub repositories and upload the notebook plus README to each.
- ■ Set Colab sharing to “Anyone with the link can view” and test both links.
- ■ Submit both projects together using the form provided by Pluto Academy.

Academic integrity: Use this guide to learn and build the projects. Run the code, inspect outputs, and write your own findings. The internship brief warns against submitting copied notebooks.